

ARTIFICIAL INTELLIGENCE

Assignment - 1

AI SMART BUS ARRIVAL PREDICTION

Submitted By

YEDHUKRISHNAN P J

Group 11

S7 CSEA

Roll No:62

AI SMART BUS ARRIVAL PREDICTION

1 Introduction

The AI-Based Smart Bus Arrival Prediction System is an intelligent agent designed to track live bus movements, predict arrival times, and assist commuters through a conversational AI interface. The system continuously processes real-time transit data and updates an interactive dashboard.

Manual tracking of bus schedules is often inaccurate due to unpredictable traffic and weather conditions. The proposed intelligent agent automates transit monitoring by applying real-time data tracking and Large Language Model (LLM) decision-making to inform users of delays and route optimizations, ensuring an efficient and stress-free commuting environment.

2 PEAS Representation

Agent Type	Performance Measure	Environment	Actuators	Sensors
AI SMART BUS ARRIVAL PREDICTION	Accurate ETA predictions Helpful AI responses Timely delay notifications	Transit network Buses & Routes Traffic conditions User queries	Display live ETAs Plot buses on map Generate chat responses	Real-time transit feed User chat input

Table 1: PEAS Representation of the AI SMART Bus Arrival Prediction

3 Environment Properties

Property	Type	Justification
Observability	Partially Observable	The intelligent agent knows the current location of the buses, but underlying traffic or weather conditions causing delays may not be fully visible.
Determinism	Stochastic	The environment involves randomness; bus speeds vary, and unpredictable delays or accidents can alter ETAs unexpectedly.
Dynamics	Dynamic	The environment changes continuously. Bus locations update and ETAs decrease while the agent is processing user queries.
Discreteness	Continuous	Bus locations (latitude/longitude), speeds, and elapsed time are continuous variables.
Agents	Single Agent	A single central Transit AI agent is responsible for tracking all buses and generating responses for the user.
Knowledge	Known	The rules governing routes, basic travel times, and map layouts are predefined and known to the agent.

Table 2: Environment Properties of the AI SMART Bus Arrival Prediction

4 Rationality Measure

A rational agent selects the action that maximizes its performance measure based on the information available. In the proposed AI-Based Smart Transit Prediction System, the intelligent agent behaves rationally by constantly polling the latest bus coordinates and mathematically updating the ETAs.

Furthermore, when the user asks a question, the agent rationally injects the *current live state* of the network into the LLM's context window. This ensures the agent never hallucinates outdated schedules, but consistently provides the most accurate and up-to-date transit advice.

5 Innovation Point

The proposed system innovates by bridging the gap between raw data dashboards and human interaction. Instead of forcing a user to stare at a map to figure out if they will be late, the system uses an intelligent LLM agent that understands the live map context.

It can be further enhanced with features such as GPS integration for the user's phone to calculate walking time to the bus stop, push notifications for sudden route changes, and integration with city traffic cameras.

6 Diagram

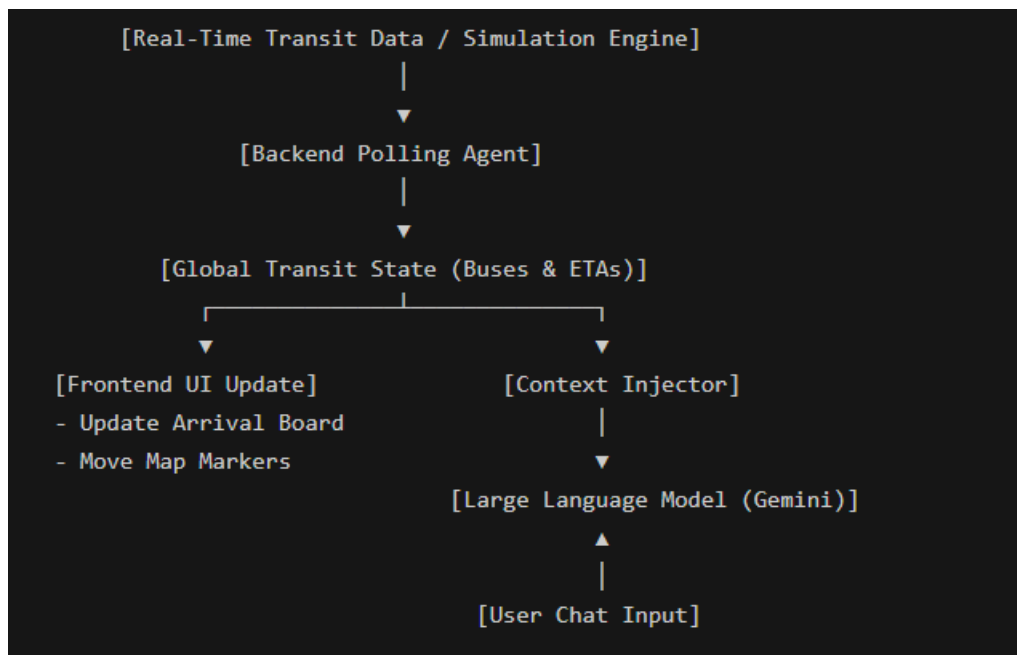


Figure 1: Architecture of the AI SMART Bus Arrival Prediction

7 Conclusion

The AI-Based Smart Bus Arrival Prediction System provides an efficient and intelligent solution for modern commuting. By combining real-time location tracking with conversational AI, the system ensures accurate ETA predictions, optimal route suggestions, and reduced commuter anxiety during unexpected delays.

8 Simulation

The simulation of the AI-Based Smart Transit Prediction System demonstrates the working of the intelligent agent in processing dynamic data. The system initializes a fleet of buses across three distinct routes. A background thread runs a simulation loop every 5 seconds, randomly advancing bus locations and decreasing ETAs to simulate real-world movement and unpredictable delays.

Simultaneously, the user can interact with the terminal or web interface. When the user types a query (e.g., "Are there any delays?"), the AI agent instantly analyse the simulated data array, identifies any buses marked as "delayed", and generates a contextual response advising the user on the best course of action.

9 Screenshots of Full Code

```
1 import os
2 import asyncio
3 import time
4 import requests
5 from typing import List, Dict, Any
6 from fastapi import FastAPI, BackgroundTasks, HTTPException
7 from fastapi.middleware.cors import CORSMiddleware
8 from pydantic import BaseModel
9 from dotenv import load_dotenv
10
11 # Optional: gtfs-realtime-bindings for parsing protobuf feeds
12 try:
13     from google.transit import gtfs_realtime_pb2
14     GTFS_AVAILABLE = True
15 except ImportError:
16     GTFS_AVAILABLE = False
17
18 # Google GenAI SDK
19 from google import genai
20 from google.genai import types
21
22 load_dotenv()
23
24 app = FastAPI(title="SmartTransit AI Backend (Production)")
25
26 app.add_middleware(
27     CORSMiddleware,
28     allow_origins=["*"],
29     allow_credentials=True,
30     allow_methods=["*"],
31     allow_headers=["*"],
32 )
33
34 # Configuration from Environment Variables
35 GTFS_URL = os.getenv("GTFS_REALTIME_URL", "https://api.example.com/gtfs-realtime")
36 GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")
37
38 if not GEMINI_API_KEY:
39     print("WARNING: GEMINI_API_KEY is not set. Chat functionality will fail.")
40
41 # Initialize Gemini Client
42 try:
43     ai_client = genai.Client(api_key=GEMINI_API_KEY)
44 except Exception as e:
45     ai_client = None
46     print(f"Failed to initialize Gemini Client: {e}")
47
48 # Global state to hold the latest parsed transit data
49 transit_state = {
50     "buses": [],
51     "last_updated": 0
52 }
53
54 # Generic Route Dictionary mapping Route IDs to display details
55 ROUTES = {
56     "R1": {"id": "R1", "name": "Downtown Express", "color": "#06B6D4"},
57     "R2": {"id": "R2", "name": "University Line", "color": "#8B5CF6"},
58     "R3": {"id": "R3", "name": "Airport Shuttle", "color": "#10B981"},
59 }
60
61 def fetch_and_parse_gtfs():
62     """
63     Fetches actual GTFS Realtime data from a transit authority API.
64     Since we don't have a specific city yet, this is a generic implementation.
65     """
66     if not GTFS_AVAILABLE:
67         print("gtfs-realtime-bindings not installed. Cannot parse feed.")
68         return
```

```

70     try:
71         response = requests.get(GTFS_URL, timeout=5)
72         if response.status_code == 200:
73             feed = gtfs_realtime_pb2.FeedMessage()
74             feed.ParseFromString(response.content)
75
76             parsed_buses = []
77             # Extract vehicles from the feed
78             for entity in feed.entity:
79                 if entity.HasField('vehicle'):
80                     vehicle = entity.vehicle
81                     # Calculate a mock ETA for UI display purposes based on stop progress,
82                     # a real app would use the TripUpdate feed for accurate ETAs.
83                     eta = 10
84
85                     parsed_buses.append({
86                         "id": str(vehicle.vehicle.id) if vehicle.HasField('vehicle') else "Unknown",
87                         "routeId": str(vehicle.trip.route_id) if vehicle.HasField('trip') else "Unknown",
88                         "location": 50, # In a real map, use vehicle.position.latitude and longitude
89                         "eta": eta,
90                         "status": "on-time",
91                         "aiInsight": "Live tracked vehicle."
92                     })
93
94             # Update global state
95             transit_state["buses"] = parsed_buses
96             transit_state["last_updated"] = time.time()
97             print(f"Updated live feed with {len(parsed_buses)} vehicles.")
98         else:
99             print(f"Failed to fetch GTFS feed: {response.status_code}")
100     except Exception as e:
101         print(f"Error fetching GTFS feed: {e}")

```

```

103 async def polling_task():
104     """Background task to poll the Transit API periodically."""
105     while True:
106         # We wrap the synchronous request in a thread so it doesn't block FastAPI
107         await asyncio.to_thread(fetch_and_parse_gtfs)
108         await asyncio.sleep(30) # Poll every 30 seconds
109
110 @app.on_event("startup")
111 async def startup_event():
112     """Start the background polling when the server starts."""
113     asyncio.create_task(polling_task())
114
115 # --- API Models ---
116 class ChatRequest(BaseModel):
117     message: str
118
119 class ChatResponse(BaseModel):
120     reply: str
121
122 # --- API Endpoints ---
123 @app.get("/api/buses")
124 async def get_buses():
125     """Returns the real-time state parsed from the API."""
126     # Since this is a template, if we have no buses (e.g. invalid URL), we will return a fallback array
127     # just so the frontend doesn't crash while you are setting up your actual API.
128     buses = transit_state["buses"]
129     if not buses:
130         buses = [
131             {"id": "DEM01", "routeId": "R1", "location": 30, "eta": 5, "status": "on-time", "aiInsight": "Demo bus. Add real GTFS URL."}
132         ]
133
134     return {"buses": buses, "routes": list(ROUTES.values())}
135

```

```

136 @app.post("/api/chat", response_model=ChatResponse)
137 async def chat(request: ChatRequest):
138     """Actual LLM endpoint using Google Gemini API."""
139     if not ai_client:
140         raise HTTPException(status_code=500, detail="Gemini API is not configured.")
141
142     buses = transit_state["buses"]
143
144     # Create context from live bus data to inject into the prompt
145     context = "Current Live Bus Network State:\n"
146     if buses:
147         for b in buses:
148             route = ROUTES.get(b["routeId"], {}).get("name", "Unknown Route")
149             context += f"- Bus {b['id']} on {route} is arriving in {b['eta']} mins. Status: {b['status']}. \n"
150     else:
151         context += "No active buses found in the network at this moment.\n"
152
153     # Construct the final prompt
154     system_instruction = (
155         "You are a helpful Transit AI Assistant. Use the provided real-time "
156         "bus network state to accurately answer the user's questions about ETAs, "
157         "delays, and routes. Keep answers concise."
158     )
159
160     prompt = f"System: {system_instruction}\n\nContext: \n{context}\n\nUser Question: {request.message}"
161
162     try:
163         response = ai_client.models.generate_content(
164             model='gemini-2.5-flash',
165             contents=prompt,
166         )
167         return ChatResponse(reply=response.text)
168     except Exception as e:
169         print(f"LLM Error: {e}")
170         raise HTTPException(status_code=500, detail="Failed to communicate with LLM.")

```

10 Output

